Jenkins

Phase 1: Spin Up the Infrastructure (Terminal)

Open your terminal or command prompt and run these three commands to get the containers running together on the same Docker network.

Bash
1. Create the isolated network
docker network create cicd-net

2. Start SonarQube
docker run -d --name sonar --network cicd-net -p 9000:9000 sonarqube:lts-community

3. Start Jenkins
docker run -d --name jenkins --network cicd-net -p 8080:8080 jenkins/jenkins:lts

(Wait about 1 minute for both applications to boot up completely in the background).

Phase 2: Grab the Security Token from SonarQube

1. Open your browser and go to **`http://localhost:9000`**.
2. Log in with Username: `admin` and Password: `admin`. (It will ask you to change the password immediately—change it to something easy like `admin123`).
3. Go to **Administration** > **Security** > **Users**.
4. Click on the **Tokens** icon next to the admin user.
5. Enter a name (e.g., `my-token`), click **Generate**, and **copy the long string of letters and numbers**. You will need this in the next phase!

Phase 3: Prepare Jenkins

1. Open a new tab and go to **`http://localhost:8080`**.
2. To unlock Jenkins, go back to your terminal and run this command to get the password:
3. Bash

docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

4.
5.
6. Paste that password into Jenkins, select **"Install suggested plugins"**, and create an admin user when prompted.
7. **Install the Scanner Plugin:** * Go to **Manage Jenkins** > **Plugins** > **Available plugins**.
    - Search for `SonarQube Scanner` and install it. Restart Jenkins if prompted.
8. **Save your Token:**
    - Go to **Manage Jenkins** > **Credentials** > **System** > **Global credentials** > **Add Credentials**.

    - **Kind:** Select *Secret text*.
    - **Secret:** Paste the token you copied from SonarQube.
    - **ID:** Type `sonar-token`. Click **Create**.
9. **Link the SonarQube Server:**
    - Go to **Manage Jenkins** > **System**.
    - Scroll down to *SonarQube servers*, click **Add SonarQube**.
    - **Name:** `sonar-server` *(This must exactly match the name inside your pipeline script's `withSonarQubeEnv` block)*.
    - **Server URL:** `http://sonar:9000` *(We use `sonar` because they are on the same Docker network)*.
    - **Server authentication token:** Select the `sonar-token` you created. Click **Save**.
10. **Configure the Scanner Tool:**
    - Go to **Manage Jenkins** > **Tools**.
    - Scroll down to *SonarQube Scanner*, click **Add SonarQube Scanner**.
    - **Name:** `sonar-scanner` *(This must exactly match the name inside your pipeline script's `tool` block)*.
    - Check **Install automatically**. Click **Save**.

Phase 4: Create and Execute the Job

1. On the Jenkins dashboard, click **New Item**.
2. Give it a name (e.g., `Exam-Project`), select **Pipeline**, and click **OK**.
3. Scroll down to the **Pipeline** script section and paste your exact script with the spelling fix and the negative report setup:

```
Groovy
pipeline {
    agent any
    stages {
        stage('Prepare Workspace') {
            steps {
                sh "echo 'password = \"SuperSecret123\"' > helloworld.py"
            }
        }
        stage('Code Analysis') {
            environment {
                scannerHome = tool 'sonar-scanner'
            }
            steps {
                script {
                    withSonarQubeEnv('sonar-server') {
                        sh """
                            ${scannerHome}/bin/sonar-scanner \
                            -Dsonar.projectKey=your_project_key \
                            -Dsonar.projectName=your_project_name \
                            -Dsonar.projectVersion=1.0 \
```

```
                    -Dsonar.sources=.
                """
            }
        }
    }
  }
}
```

4. Click **Save**.
5. Click **Build Now** on the left menu.

Maven & Gradle installation

Let's knock out this next program! Both Maven and Gradle are build automation tools—they take your raw source code, download the required dependencies, compile everything, run your tests, and package it into a deployable artifact (like a `.jar` or `.war` file). Think of them as the automated assembly lines for Java projects.

Since I am an AI, I don't have to take exams, but I do know how to compress and structure data for maximum human retention. Here is your cheat sheet to memorize their fundamentals and setups.

Maven vs. Gradle Fundamentals

Memorize this table to easily contrast the two during your exam:

Feature	Apache Maven	Gradle
Configuration File	`pom.xml`	`build.gradle`
Language Used	XML	Groovy or Kotlin DSL
Core Concept	Strict, predefined build phases (Lifecycle)	Flexible, dependency-based Task Graph
Performance	Standard speed	Highly optimized (uses incremental builds & caching)
Best For	Standardized, traditional Java projects	Large, complex projects & Android development

The Golden Prerequisite

Before installing either tool, you must have Java installed. Both Maven and Gradle run on the Java Virtual Machine (JVM).

1. Install the Java Development Kit (JDK).
2. Set the `JAVA_HOME` environment variable to your JDK folder.
3. Verify it in your terminal:
4. `java -version`

Maven Installation Setup

The setup for both tools relies on the exact same logic: **Download -> Extract -> Add to PATH.**

1. **Download:** Get the binary zip archive (`apache-maven-x.x.x-bin.zip`) from the official Maven website.
2. **Extract:** Unzip the folder to a permanent location on your drive (e.g., `C:\maven` on Windows or `/opt/maven` on Linux).
3. **Configure PATH:** Copy the path to the extracted `bin` folder (e.g., `C:\maven\bin`) and paste it into your system's `PATH` environment variable.
4. **Verify:** Open a fresh terminal and run the version command:
5. `mvn -v`

Gradle Installation Setup

Notice how the steps mirror Maven completely. If you remember one, you remember both.

1. **Download:** Get the binary-only zip file (`gradle-x.x-bin.zip`) from the official Gradle website.
2. **Extract:** Unzip the folder to a permanent location on your drive (e.g., `C:\gradle` on Windows or `/opt/gradle` on Linux).
3. **Configure PATH:** Copy the path to the extracted `bin` folder (e.g., `C:\gradle\bin`) and paste it into your system's `PATH` environment variable.
4. **Verify:** Open a fresh terminal and run the version command:
5. `gradle -v`

Final Exam Tip: If your professor asks how to create a new project from scratch using these tools, memorize these two generation commands:

- **Maven:** `mvn archetype:generate`
- **Gradle:** `gradle init`

Maven project setup and pom.xml structure

You are crushing it. Breaking down Maven into these three core concepts (POM, Dependencies, Plugins) is exactly how you master it. For your exam, think of Maven as a kitchen: the **POM** is your recipe, the **Dependencies** are your ingredients, and the **Plugins** are your appliances.

Here is your heavily condensed cheat sheet to memorize for tomorrow.

1. Create a Maven Project (The Setup)

To create a standard Java project structure instantly, you just need to remember one command.

- **The Command:** > `mvn archetype:generate`
- **What happens:** Maven will ask you for a `groupId` (your package name, e.g., `com.exam`), an `artifactId` (your project name, e.g., `my-app`), and a version. It then generates the standard directory structure (`src/main/java`, `src/test/java`) and a basic `pom.xml` file.
- mvn archetype:generate -DgroupId=com.example -DartifactId=demo-project -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
-

2. Understanding the POM File (The Recipe)

POM stands for **Project Object Model**. It is an XML file (`pom.xml`) located at the root of your project. It is the brain of Maven and tells it exactly how to build your project.

The 3 Core "Coordinates" to memorize:

Every POM file identifies the project using these three tags:

1. `<groupId>`: The organization or package name (e.g., `com.college.exam`).
2. `<artifactId>`: The name of your specific project (e.g., `hello-world-app`).
3. `<version>`: The current version of your code (e.g., `1.0-SNAPSHOT`).

3. Dependency Management (The Ingredients)

A "Dependency" is simply an external library (like JUnit for testing or Spring for web development) that your code needs to work. Instead of manually downloading JAR files, you list them in the POM, and Maven automatically downloads them from the **Maven Central Repository**.

The XML to Memorize:

Dependencies are wrapped inside a `<dependencies>` block. Here is the classic JUnit example:

```
XML
  dependencies
    dependency
      groupId          groupId
      artifactId         artifactId
      version          version
      scope          scope
     dependency
  dependencies
```

4. Plugins (The Appliances)

Maven itself is just an empty framework; it doesn't actually know how to compile code or run tests. It delegates all the actual work to **Plugins**. For example, the `maven-compiler-plugin` compiles your Java code, and the `maven-surefire-plugin` runs your tests.

The XML to Memorize:

Plugins are configured inside a `<build>` block.

```
XML
  build
    plugins
      plugin
        groupId                          groupId
        artifactId                     artifactId
        version          version
       plugin
     plugins
   build
```

Quick Exam Summary

If you are asked to explain the flow tomorrow, just write:

"Maven uses the `pom.xml` to identify the project, downloads required external libraries using `<dependencies>`, and executes build tasks (like compiling and packaging) using `<plugins>`."

Maven -> Gradle Java app

Phase 1: Build and Run with Maven

1. Generate the Project Open your terminal and run the generation command we memorized earlier.

> `mvn archetype:generate` *(Press Enter through the prompts to use defaults. When asked, set* `groupId` *to* ***com.exam*** *and* `artifactId` *to* ***my-app****).*

2. The Java Code Maven automatically creates a `Hello World` file for you at `src/main/java/com/exam/App.java`. It looks like this:

Java
```
package com.exam;

public class App {
    public static void main(String[] args) {
        System.out.println("Hello World from Maven!");
    }
}
```

3. Build and Run Navigate into your project folder (`cd my-app`) and run this single command to compile the code and execute the main class:

```
mvn clean compile exec:java -Dexec.mainClass="com.exam.App"
```

- **`clean`**: Deletes old compiled files.
- **`compile`**: Converts your `.java` files to `.class` files.
- **`exec:java`**: Tells Maven to run the program.

Phase 2: Migrate to Gradle (The Magic Trick!)

Here is the absolute best trick for your exam tomorrow: **You do not have to write the Gradle file from scratch.** Gradle has a built-in feature to read a `pom.xml` and automatically convert it into a `build.gradle` file!

1. The Migration Command Make sure you are still inside your `my-app` folder (where the `pom.xml` is). Run:

```
gradle init
```

2. Answer the Prompts Gradle will detect the Maven project. It will ask you:

- *Found a Maven build. Generate a Gradle build from this?* **(Type 'yes')**
- *Select build script DSL:* **(Choose '1' for Groovy)**
- *Generate build using new APIs?* **(Type 'yes' or 'no', either is fine for the exam)**

3. Verify the Migration Type `ls` or `dir` in your terminal. You will see that Gradle has automatically generated a **`build.gradle`** file alongside your old `pom.xml`.

4. Run the App with Gradle To run the exact same Java application using Gradle, you just need to add two tiny lines to your newly generated `build.gradle` file so Gradle knows where your "main" method is.

Open `build.gradle` and ensure it has the `application` plugin and the `mainClass`:

Groovy
```
plugins {
    id 'java'
    id 'application' // Add this
}

application {
    mainClass = 'com.exam.App' // Add this
}
```

Now, run the app using Gradle:

```
gradle run
```

Exam Summary to Memorize

- **Maven Run:** `mvn clean compile exec:java -Dexec.mainClass="com.package.Name"`
- **Migrate:** Run `gradle init` inside the Maven project folder and say "yes" to the auto-conversion.
- **Gradle Run:** Add `id 'application'` and the `mainClass` to `build.gradle`, then execute `gradle run`.

Docker microservice

Phase 1: The Design (The Code & Dockerfile)

Create a new folder for your exam project and add these three small files.

1. `app.py` (The Python Microservice) This tiny script runs a web server and connects to the database.

Python
```
from flask import Flask
import redis

app = Flask(__name__)
# The magic happens here: it connects to 'redis' by its service name!
cache = redis.Redis(host='redis', port=6379)

@app.route('/')
def hello():
    count = cache.incr('hits')
    return f"Exam Microservices! Page views: {count}"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

2. `requirements.txt` (The Dependencies) Tells Python what it needs.

Plaintext
```
flask
redis
```

3. `Dockerfile` (The Blueprint for your Python App) Tells Docker how to build the Python microservice.

Dockerfile
```
# 1. Use Python
FROM python:3.9-alpine
# 2. Set working directory
WORKDIR /code
# 3. Copy dependencies and install them
COPY requirements.txt .
RUN pip install -r requirements.txt
# 4. Copy the code and run it
COPY app.py .
CMD ["python", "app.py"]
```

Phase 2: The Deployment (Docker Compose)

Now for the actual "architecture" part. Instead of running multiple `docker run` commands manually, we write one YAML file that defines our entire infrastructure.

Create a file named **`docker-compose.yml`** in the same folder:

YAML

```
version: '3'
services:
  # Microservice 1: The Web Frontend
  web:
    build: .
    ports:
      - "5000:5000"

  # Microservice 2: The Database Backend
  redis:
    image: "redis:alpine"
```

- **Exam Key Concept:** Notice how we didn't specify a network? Docker Compose automatically creates an isolated network for these two services so they can securely talk to each other.

Phase 3: Manage (The 4 Commands to Memorize)

To complete the exam requirement, you need to execute and manage the architecture using your terminal. Navigate to the folder containing your files and use these commands:

- **1. Deploy & Start Everything:**
  `docker-compose up -d` *(The `-d` runs it in the background so you can keep using your terminal). Go to `http://localhost:5000` in your browser. Refresh it a few times to see the microservices updating the counter!*
- **2. View Running Services:**
  `docker-compose ps` *(Shows you that both `web` and `redis` containers are healthy and running).*
- **3. Check the Logs:**
  `docker-compose logs web` *(Shows you the internal terminal output of just the web microservice).*
- **4. Tear Down the Architecture:**
  `docker-compose down` *(Gracefully stops and deletes both containers and the isolated network).*

Docker Containerization

Step 1: Create the Application Files

Create a new folder and add these two files:

1. `index.html` (Your App Code)

HTML
<h1>Hello from my Exam Container!</h1>

2. `Dockerfile` (The Instructions)

Dockerfile
Use a lightweight web server OS
FROM nginx:alpine

Copy your HTML file into the container's web folder
COPY index.html /usr/share/nginx/html/index.html

Step 2: Build the Image

Open your terminal in that folder and run the build command. The `.` at the end tells Docker to look in the current directory for the Dockerfile.

```
docker build -t my-web-app .
```

- *What happens:* Docker reads the Dockerfile, downloads Nginx, inserts your HTML file, and packages it into a local image named `my-web-app`.

Step 3: Create the Container (The Missing Step!)

Now we take that image and bring it to life as a running container.

```
docker run -d --name my-running-app -p 8080:80 my-web-app
```

- **`-d`**: Runs it in the background (detached).
- **`--name`**: Gives your container a specific name (`my-running-app`) so it's not a random string of letters.
- **`-p 8080:80`**: Maps port 8080 on your computer to port 80 inside the container.

Test it: Open your browser and go to `http://localhost:8080`. You will see your "Hello from my Exam Container!" message!

Phase 4 (Recap): Now you Push!

Now that you have successfully created and tested the container, you would proceed to Docker Hub just like we covered before:

1. `docker login`
2. `docker tag my-web-app your_username/my-web-app:v1`
3. `docker push your_username/my-web-app:v1`